

Intelligent Forwarding, Transparent Redirection: SDN-Based Secure Traffic Control Using Mininet and Ryu

1st Jinyu Cai
2361409

2nd Jiayi Chu
2361209

3rd Zhiying Tang
2360010

4th Jiayu Wang
2362046

5th Xinyan Xue
2361538

Abstract—This project employs the Mininet and the Ryu controller framework to construct a software-defined networking (SDN) environment and explore controller traffic management. The work includes building a SDN topology, implementing flow-based forwarding logic, and enabling transparent redirection of client-to-server traffic. Core tasks involve installing and managing flow-table rules, inspecting packet behavior, and evaluating connection latency under forwarding and redirection conditions. We use diagrams and algorithms to show system’s design and implementation, and verify system correction through experiment. Finally, we conclude the potential directions in future development.

Index Terms—Mininet, SDN, Ryu Controller, Forwarding, Redirecting, TCP/IP, MAC

I. INTRODUCTION

A. Background

SOFTWARE Defined Network (SDN) is a modern network that separates the control plane from the data plane, and then achieving centralized management of network behavior. SDN is not directly let control logic into a single forwarding device, but introduces a logically centralized controller that helps maintain a global view of the network and use program style to configure switches. This separation helps enhance the flexibility of the network, simplifies policy deployment, and enables operators to optimize routing and resource allocation in real time. By abstracting the details of the underlying infrastructure and programmable interfaces, SDN effectively overcomes the shortcomings of traditional distributed network architectures to support the rapid innovation of network applications and services [1].

B. Task Specification

To be more specific, the project’s task can be divided in to 5 parts:

- 1) Build an SDN network topology in Mininet with the correct IP and MAC configurations for Client, Server1, and Server2.
- 2) Develop a Ryu controller that enables host reachability and installs flow entries.
- 3) Deploy the provided Client and server socket programs on the Mininet hosts to generate application-level traffic after flow entries expire.
- 4) Implement a controller logic that installs forwarding rules triggered by the first TCP SYN and measure the handshake latency.

- 5) Program a controller that redirects Client-to-Server1 traffic to Server2 and then measure the handshake latency for redirected connection.

C. Project challenges

A key challenge of this project is to enable the SDN controller correctly interpret real-time Packet_In events and translate them into flow-table operations. And the process need to ensure forwarding and redirection behaviors remain transparent to the Client. This is because the controller must dynamically create, modify, and remove flow entries based on TCP SYN packets, any wrong configuration risks will affect or break the TCP handshake. Moreover, in the redirection behavior, we need to redirect traffic from Server1 to Server2 without notifying the Client of the changes and retaining the end-to-end semantics, which brings additional complexity.

D. Practice Relevance

The redirection methods we implemented is actually a prototype for secure traffic control. By forwarding traffic from Server1 to Server2 without the client knowing, we actually built a basic “honeypot” environment. We refer to Wang and Du [2] suggested this concept, they suggest using this hidden redirection to isolate attackers while keeping the main system safe. In addition, our project also verified the idea of Moving Target Defense (MTD). And Abdelkhalek et al. [3] pointed out, changing network paths dynamically can effectively stop DoS attacks. Our experiments have proved that this kind of routing mutation is feasible in the actual SDN environment.

E. Contribution

By completing the tasks of this project, our group successfully applied the SDN principles to actual network scenarios and developed practical and feasible solutions through experiments. Firstly, we built a SDN topology in Mininet, configuring the host addressing, and ensured a reliable connection between the server and the Client as the foundation for later tasks. Moreover, we implemented the controller application by using the Ryu framework, implementing the forwarding logic to support transparent traffic redirection behavior. In addition, we conducted latency analysis using Wireshark to compare forwarding and redirection behaviors, providing deep insights into SDN performance. Overall, by completing the tasks of this project, our understanding of SDN and the traffic control technical have been enhanced.

II. RELATED WORK

In Software-Defined Networking(SDN), the ability to redirect traffic has attracted significant research interest for both security and performance enhancement. In security aspect, Abdelkhalek et al. [3] proposed a Moving Target Defense(MTD) strategy that dynamically adjust the routing path to avoid potential attackers . Although our implementation performs static server redirection, it uses the same SDN flow modification capability, providing a foundation for future MTD extensions. In addition, Du and Wang applied game theory to SDN honeypot design [2], which created intelligently redirect suspicious traffic systems. Our work follows this transparent traffic redirection concept, where the traffic is redistributed without the client devices knowing. Moreover, performance research can provide more deep insights. Khalili et al. [4] demonstrated that flow setup latency grows with network scale, which aligns with our finding that controller processing during redirection introduces measurable delay. Additionally, Appelman et al. highlighted performance differences between software-emulated and hardware-based OpenFlow switches [5], a relevant caveat since our experiments use Mininet for functional validation rather than performance measurement. By building a minimal SDN redirection testbed, our project demonstrates these core principles in practice while establishing reproducible benchmarks for educational purposes.

III. DESIGN

A. Architecture of SDN Network

The SDN network consists of a controller, an SDN switch, and three hosts (Client, Server1, Server2). And the separation of the control plane and the data plane is achieved through the OpenFlow protocol. The switch is responsible for forwarding based on the flow table, the controller is responsible for handling Packet_In events, installing flow table entries, and determining the way data forwarding or redirecting. The structure of network topology is shown in Fig.6.

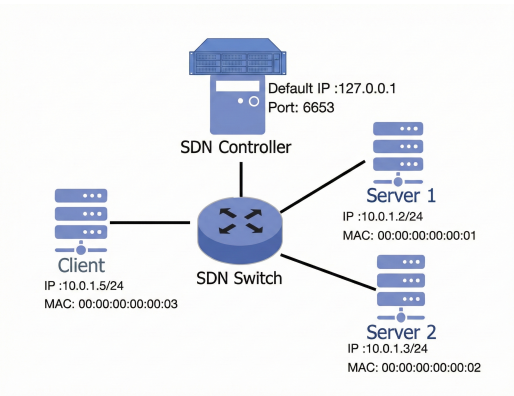


Fig. 1. The Network Topology

Here is the explanation of the three core components in SDN network:

- 1) SDN Controller: As the centralized control core, the controller mainly communicates with the switch through the OpenFlow protocol. Its functions include parsing Packet_In events, determining the forwarding/redirecting strategy, distributing Flow_Mod, and dynamically managing the flow table by using `idle_timeout`.
- 2) SDN Switch: The SDN switch executes controller-issued actions, such as forward, redirect, flood, and forwards data based on flow table entries. If no match is found, it sends a Packet_In message to the controller, waiting a Packet_Out message for further processing.
- 3) Hosts: The Client will initiate a TCP connection, and Server1 and Server2 will receive these connections and provide services for the Client. The hosts verify forwarding and redirection operations through socket programs, and the communication between them by switches, which are uniformly managed by the SDN controller.

B. Workflow of the project

1) *Forwarding Function:* The forwarding controller installs specific flow entries for TCP connections in (`ryu_forward.py`) . When the Client sends TCP SYN to Server1, Switch will send Packet_In to controller. The controller uses `_is_client_syn_to_server1()` to detect the SYN, which resolves Server1's port from static mapping and installs a flow entry though Flow-Mod. The entry matches Client→Server1 traffic and forward to port 2. The SYN is forwarded via Packet_Out. All subsequent packets match this entry. Flow entry includes `idle_timeout=5` as required. The controller also install a reverse flow for Server1→Client traffic, which helps to avoid unnecessary events from Packet_In return traffic.

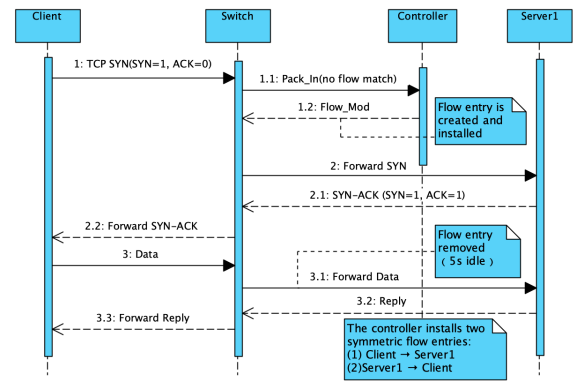


Fig. 2. Sequence diagram of forwarding

2) *Redirecting Function:* The redirecting controller (`ryu_redirect.py`) implements transparent traffic redirection. When Controller detects TCP SYN from Client to Server1, it installs two flow entries: upstream (Client→Server1 → Server2, modifying destination MAC/IP) and downstream (Server2→Client → Server1, modifying source MAC/IP). The modified SYN sent to Server2 via Packet_Out. Server1

remains idle. All subsequent packets match the entries. Both entries implement `idle_timeout=5`.

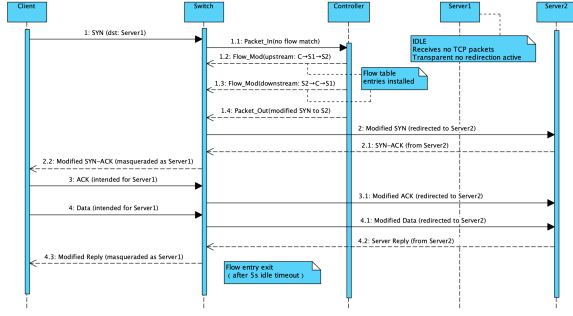


Fig. 3. Sequence diagram of redirecting

C. Algorithm

Algorithm 1 Forwarding Algorithm

Input: Packet_In msg

Output: Flow entry

```

pkt ← parse(msg)
eth ← get_eth(pkt)
src, dst ← eth.src, eth.dst
in_port ← msg.match['in_port']

if is_ipv4(pkt) ∧ is_tcp(pkt) then
    ip ← get_ipv4(pkt)
    tcp ← get_tcp(pkt)

    if ip.src="10.0.1.5" ∧ ip.dst="10.0.1.2" ∧ is_syn(tcp)
    then
        out_port ← 2
        match_fw ← match_tcp(in_port, src, dst, ip.src,
                               ip.dst, tcp)
        actions_fw ← [output(out_port)]
        add_flow(match_fw, actions_fw, idle=5, prio=10)

        match_rev ← match_tcp(in_port=2, src_mac=dst,
                               dst_mac=src, src_ip="10.0.1.2",
                               dst_ip="10.0.1.5",
                               tcp)
        actions_rev ← [output(1)]
        add_flow(match_rev, actions_rev, idle=5, prio=10)

        send_packet(actions_fw)
    return

```

```

if dst ∈ mac_table then
    out ← mac_table[dst]

```

else

```

    out ← FLOOD
    actions ← [output(out)]
    match ← match_l2(in_port, src, dst)
    add_flow(match, actions, idle=5, prio=1)
    send_packet(actions)

```

Algorithm 2 Redirecting Algorithm

Input: Packet_In msg

Output: Redirect flows

```

pkt ← parse(msg)
eth ← get_eth(pkt)
src, dst ← eth.src, eth.dst
in_port ← msg.match['in_port']

```

if $is_ipv4(pkt) \wedge is_tcp(pkt)$ then

```

    ip ← get_ipv4(pkt)
    tcp ← get_tcp(pkt)

```

if $ip.src="10.0.1.5" \wedge ip.dst="10.0.1.2" \wedge is_syn(tcp)$ then

```

    // Upstream: Client → Server1 → Server2
    match_up ← match_ip(1, "10.0.1.5", "10.0.1.2")
    actions_up ← [set(dst_mac="00:00:00:00:00:02"),
                  set(dst_ip="10.0.1.3"), output(3)]
    add_flow(match_up, actions_up, idle=5, prio=20)

```

```

    // Downstream: Server2 → Client →
    Server1

```

```

    match_down ← match_ip(3, "10.0.1.3", "10.0.1.5")
    actions_down ← [set(src_mac="00:00:00:00:00:01"),
                    set(src_ip="10.0.1.2"), output(1)]
    add_flow(match_down, actions_down, idle=5,
              prio=20)

```

```

    send_packet(actions_up)

```

return

if $dst \in mac_table$ then

```

    out ← mac_table[dst]

```

else

```

    out ← FLOOD
    actions ← [output(out)]
    match ← match_l2(in_port, src, dst)
    add_flow(match, actions, idle=5, prio=1)
    send_packet(actions)

```

IV. IMPLEMENTATION

A. Develop Environment

This project uses Oracle VM VirtualBox to build an Ubuntu virtual machine as the development environment. The details configuration is shown in the following table:

TABLE I
DEVELOPMENT ENVIRONMENT

Virtual Machine OS	Ubuntu 20.04.4
CPU	i5 (12 cores)
RAM	16GB@4800 MHz
IDE	PyCharm 2023.2.1
Python	Python 3.8.10
SDN Controller	Ryu 4.34, Wireshark

B. Programming Skills

- 1) **Object-Oriented Programming (OOP):** The implementation follows OOP principles with two controller classes, `RyuForward` and `RyuRedirect`, both extending Ryu's base `RyuApp` class. Each class manages its own state, including MAC-to-port learning tables and static host mapping. This design keeps the code modular and easier to maintain.
- 2) **Event-Driven Architecture:** The controllers use Ryu's event system through the `@set_ev_cls` decorator. Key event handlers manage two main tasks. They handle switch feature events for initial setup and also process `packet_in` events for real-time traffic handling, which help SDN control plane more efficient.
- 3) **Flow Table Management:** The `add_flow()` method helps to install dynamic flow entries, which has configure match rules, actions, priorities, and the parameter `idle_timeout=5`. This enable us to control traffic precisely and better manage switch resources.
- 4) **Robust Protocol Parsing and Exception Handling:** The controller uses Ryu's packet library to parse packets at multiple layers, which extracts the Ethernet, IPv4, and TCP headers. Methods like `_is_client_syn_to_server1()` can be used to identify TCP SYN packets accurately. It also handles some edge cases, including LLDP packet filtering and buffer management, which maintain reliable operation.

C. Steps of Implementation

- 1) **Design and implement for the Mininet Topology:** we built a SDN network in `networkTopo.py`, which consists of a switch, a remote Ryu controller, and three hosts with assigned fixed IP/MAC to ensure predictable `Packet_In` operation.
- 2) **Connect to Ryu Controller:** we added a controller in mininet that all OpenFlow messages generated by the switch can be forwarded to the Ryu controller for centralized management.
- 3) **Implement the Basic Switching Logic:** we achieved the forwarding by extending `simple_switch_13`, where the controller learns MAC addresses and installs short `idle_timeout` flow rules for Client-Server1 TCP connection.
- 4) **Development of the Redirection Mechanism:** we implemented the controller to intercept the `Packet_In` event in `ryu_redirect.py`. Then we rewrite the destination IP/MAC field to Server2 and rewrite the corresponding forwarding data packet for the return traffic to maintain transparency.
- 5) **Application-Layer Message Transmission:** The Client sends messages with number to the server continuously, which allows us to observe whether packets reach Server1 or are redirected to Server2 according to the controller's policy.
- 6) **Testing and Validation:** We confirm whether the system behavior correctly with the required specifications by

comparing the terminal outputs on Server1 and Server2, and thereby verify the correctness of forwarding and redirection.

D. Actual Implementation

- 1) **Forwarding SDN controller:** We implemented the forwarding controller in `ryu_forward.py` by extending the behavior of the traditional learning switch to support TCP stream installation. In this code, we have added a detection logic for the first TCP SYN packet of Client→Server1, which can automatically install a flow table with `priority=10` and `idle_timeout=5` for this five-tuple and immediately `Packet_Out` the original SYN. Let the subsequent traffic be forwarded through the switch hardware. In addition, we also implemented Server1 → Client flow, the controller installs it to ensure that the SYN-ACK and subsequent packets forwarded without affected any other `Packet_In` events. We verify the flow table correctly written through the output results of `ryu-manager` and `dpctl dump-flows`, and prove the connection stability through the output of the Client/Server1/Server2 terminal.
- 2) **Redirecting SDN controller:** We implemented transparent service redirection based on the NAT logic of OpenFlow `OFPACTIONSetField` in `ryu_redirect.py` using the OpenFlow header. After receiving the first TCP SYN packet destined for Server1, the controller identifies this flow as a candidate flow for redirection and installs two NAT-style flow entries. To ensure that the TCP connection is not interrupted, we have to deceive the Client, so we installed two flow tables. The upstream rule matches data packets with Client as the source and Server1 as the target. Before forwarding the data packet to the corresponding switch port, it rewrites the destination MAC and IP address to the MAC and IP address of Server2. On the contrary, the downstream rules match the data packets with Server2 as the source and the Client as the destination. It rewrites the source MAC and IP address back to the values of Server1 to ensure that the Client believes it is communicating with Server1. This bidirectional rewrite ensures that the TCP handshake can be completed completely and without interruption. Therefore, our implementation ensures strict follow with TCP/IP protocol stack requirements that guarantees reliable connections, and while successfully hides Server2 from the client.

E. Difficulties and Solutions

During our tests, we found that repeatedly using Wireshark for packet capture gradually slow down the network performance. After several consecutive test runs, we noticed the delay increased significantly, which affected the stability of the results. To address this slowdown problem, we adopted two steps. Firstly, we run the `mn -c` command before starting each test, which will clear any remaining network topology and flow table entries. And then, after each packet capture session

ended, we restart Wireshark. This help to release memory buffers that had built up during captures, which prevented memory exhaustion.

In addition, we also faced the challenge of identifying and removing invalid test data. Invalid cases are typically showing that client-side errors like “No route to host” or “Connection refused,” some abnormal TCP handshake retransmissions with no SYN-ACK response, connection timeouts, or incomplete three-way handshakes visible in Wireshark. To ensure our data accurate and reliable, we used the filtering rule: only test runs that completed a full TCP three-way handshake and displayed none of the above issues were viewed as a valid data and that can used in our test.

V. TESTING AND RESULTS

A. Testing Environment & Metrics

TABLE II
TESTING ENVIRONMENT

Virtual Machine OS	Ubuntu 20.04.4
CPU	i9-12950HX
RAM	64GB@4800 MHz
IDE	PyCharm 2024.2
Python	Python 3.8.10

B. Testing Steps

- Task 1:** Open a terminal and run `sudo python3 networkTopo.py`. This action will start the Mininet environment and open the 3 XTerm windows for the Client, Server1, Server2, and switch. In the terminal of each host, we execute `ifconfig` to verify whether the assigned IP and MAC addresses match the predefined values in `networkTopo.py`. If their match, it will confirm that the topology has been correctly created and all hosts have been correctly initialized.

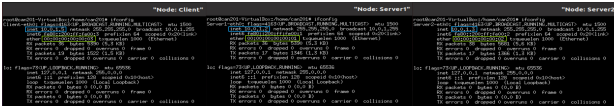


Fig. 4. Task1 screenshots

- Task 2:** First, start the controller by running `sudo ryu-manager ryu_forward.py`. Next, we return to the Mininet terminal and execute `pingall` to verify the connections between all hosts. Then, we execute `sudo ovs-ofctl -O OpenFlow13 dump-flows s1` on the switch terminal to check if there is an ICMP message triggering the flow installation. After 5 seconds, we dumped the flow table again to confirm that the entry had expired. This will verify whether the controller has been correctly installed and the flow rules have been removed.

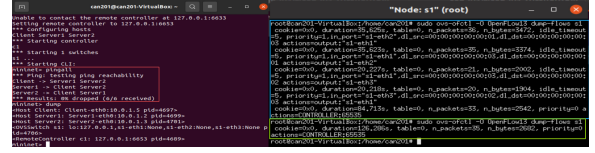


Fig. 5. Task2 screenshots

- Task 3:** Restart Mininet and the controller (in `networkTopo.py` and `ryu_forward.py`). Start the server on Server1 and running `python3 server.py` on Server2, and wait for at least 5 seconds to ensure that previous stream entries have expired. Then run `python3 client.py` on the Client side. We observed that Server1 prints the received messages to confirm that the TCP data packets sent to Server1 have been correctly forwarded. By rechecking the flow table of the switch, we verified that the forwarding rules for this TCP flow have been installed.

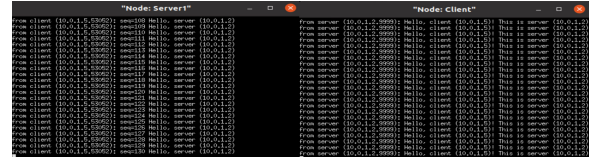


Fig. 6. Task3 screenshots

- Task 4:** Before running the Client, open a new terminal and use `sudo tcpdump -i any tcp port 9999 -w handshake_forward.pcap` to capture data packets from the Client interface. Then, repeat the above steps: run `server.py` on both servers, wait for 5 seconds, and then start `client.py`. Next, obtain the captured `.pcap` file in Wireshark. By analyzing the timestamp difference between the SYN and ACK messages, measure the three-way handshake delay under normal TCP forwarding.

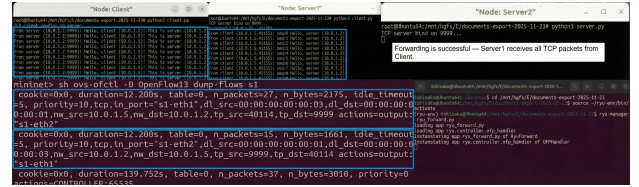


Fig. 7. Task4.1 screenshots

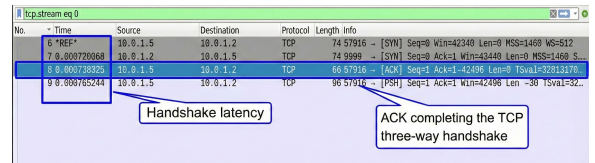


Fig. 8. Task4.2 screenshots

- Task 5:** Firstly, start the redirect controller by running the `sudo ryu-manager ryu_redirect.py` command. Next, start `server.py` on Server1 and Server2, wait for 5 seconds, and then run `client.py` from the Client host. As shown in Fig.9, all the messages initially sent to

Server1 were redirected to Server2, and Server2 printed out the received data. However, the Client still believes it is communicating with Server1, which verifies the correction of the IP and MAC rewrite logic. And then, we repeated the packet capture operation and used Wireshark to analyze the handshake latency. Finally, we compared these results with those forwarded in Task 4 to analyze the specific impact of package-level redirection on performance.

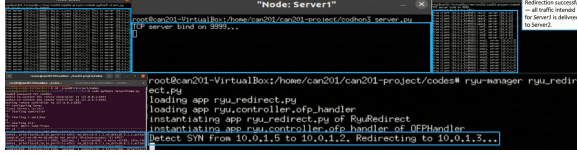


Fig. 9. Task5.1 screenshots

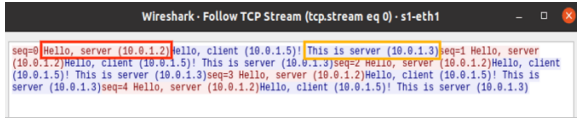


Fig. 10. Task5.2 screenshots

C. Testing result

In conclusion, we successfully establishes the correct network topology, which ensure full node connectivity, accurately packet forward and transparent redirection behavior through the SDN switch. By examining TCP three-way handshake time, we observed that 99.58% of the latency occurs during the first two steps. This is because when the switch initial contact with the controller, which need some time to wait for instructions and install the appropriate forwarding rules and it will occur some delays. In contrast, since necessary flow entries are already in place, which enable forwarding to proceed nearly two orders of magnitude, so the final handshake accounts for only 0.42%.

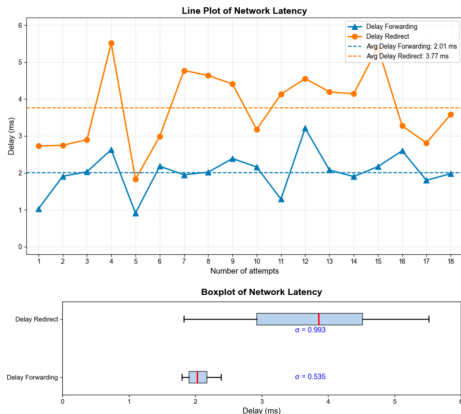


Fig. 11. Network Latency Test Statistics

As shown in Fig. 11, we performed 18 valid tests for both forwarding and redirection. After excluding incomplete TCP handshakes, we calculate the average latency and standard

deviation. The results shows that the average latency for redirection is about 1.8ms higher than forwarding, and we also observed the redirection has greater standard deviation. The difference may come from some extra time that takes to change the packet's destination address. And this issue also make forwarding times are more variability. Further automated testing is necessary to validate this hypothesis and reduce the impact of network fluctuations and other external factors.

VI. CONCLUSION

This project developed an SDN-based secure traffic control system, which has the capability of directly forwarding and transparent client-to-server redirection. By using the Mininet topology and two Ryu controller applications, we achieved dynamic flow installation triggered by TCP SYN packets and NAT-based redirection logic, without the client awareness. The use of bidirectional flow table entries enhances the stability of the connection and reflects a more complete SDN forwarding design. Test results show that although redirection causes slightly higher latency due to additional head rewriting, both mechanisms operate stably and correctly under all test conditions, verifying the reliability of the controller logic and the correctness of the flow rules. However, one of the limitation of this experience is that the controller processes flows on each basis connection, which may not have scalability or efficiently under real-world heavy or dynamic traffic.

For future works, we want to enhance the system's robustness by optimizing the flow table update strategy, minimizing controller overhead, and preventing flooding under high traffic conditions. These improvements can help us improve the scalability and applicability of SDN traffic control in actual network scenarios.

VII. ACKNOWLEDGMENT

All the members of our group have completed their tasks carefully and keep effective communication during the progress. Therefore, we totally agree that each of us has contributed equally to this project.

REFERENCES

- [1] N. McKeown, T. Anderson, H. Balakrishnan, G. Parulkar, L. Peterson, J. Rexford, S. Shenker, and J. Turner, "Openflow: Enabling innovation in campus networks," *ACM SIGCOMM Computer Communication Review*, vol. 38, no. 2, pp. 69–74, 2008.
- [2] M. Du and K. Wang, "An SDN-enabled pseudo-honeypot strategy for distributed denial of service attacks in Industrial Internet of Things," *IEEE Transactions on Industrial Informatics*, vol. 16, no. 1, pp. 648–657, 2020.
- [3] M. Abdelkhalik, B. Hyder, M. Govindarasu, and C. G. Rieger, "Moving target defense routing for SDN-enabled smart grid," in *2022 IEEE International Conference on Cyber Security and Resilience (CSR)*. IEEE, 2022, pp. 215–220.
- [4] R. Khalili, Z. Despotovic, and A. Hecker, "Flow setup latency in SDN networks," *IEEE Journal on Selected Areas in Communications*, vol. 36, no. 12, pp. 2631–2639, 2018.
- [5] M. Appelman and M. de Boer, "Performance analysis of OpenFlow hardware," University of Amsterdam, Tech. Rep., Feb 2012.